

tass — a Troll Farm bot

CodinGame Spring Challenge 2026 · Silver league rank **1 / 681** (score 23.95 peak) · version **v1.1.5-nolock** · 2026-07-02

1. The game in one paragraph

Two players farm a point-symmetric grid (8–11 rows, width = 2×height) for 300 turns. Trolls harvest fruit from trees (1 pt each), chop trees for **wood (4 pts each — the dominant currency)**, mine iron, plant new trees, and train more trolls. Training the n -th troll costs $n + \text{stat}^2$ of a resource per stat: PLUM→speed, LEMON→carry, APPLE→harvest, IRON→chop. A felled tree yields $\min(\text{size}, \text{free carry})$ wood and is gone. Trees adjacent to water grow dramatically faster (APPLE cooldown 9→2, PLUM 8→3, BANANA 6→4). Only banked resources score; carried goods are worth nothing at the buzzer.

2. The opponent: Boss 4, decoded

Promotion to Gold requires out-rating **Boss 4**, which sits *above the #1 human player* in Silver — so the real bar is "be the best bot of 681". We reverse-engineered the boss from full debug dumps (`cgauto/last_console.txt` , per-turn `@TFD/@TFSUM` state) and arena replays. Its fixed script:

- **t≈2**: trains a cheap (1,1,1,2) utility troll; the starter plants a base **LEMON** orchard and mines iron.
- **Harvesting is strictly local**: its starter never left radius 5 of its shack in 300 turns (measured); the utility troll averages 4.7.
- **Hoards to lemon 18**, then at **t≈150** trains one (2,4,2,2) power-chopper — carry 4 means 16 pts per felled size-4 tree.
- That chopper then **CHOPs every single turn**, raiding deep into our half; boss wood is ≈0 before t≈190, then ~1 wood / 5 turns.
- Never trains a 4th troll (ends games with 20+ lemon unspent).

This script is modelled as a sparring partner in `rust/src/strategies/script_boss.rs` (`scriptboss`), alongside the older greedy-expansion model `silver_boss.rs`. **Every change must hold on both models** — the rule that caught several would-be regressions (§6).

3. The core idea: a task-market scheduler with denial camping

Instead of fixed per-troll roles, every turn is an auction. For each (troll, task) pair we compute a *marginal rate* — points per turn — and greedily assign the best pairs under exclusivity. Strategy emerges from the prices, not from if-else role code.

Implemented twice, kept in sync: `rust/src/main.rs :: decide_sched()` (the single-file CodinGame submission) and `rust/src/strategies/sched_bot.rs` (the simulator mirror with env-var tuning knobs). Task types (enum `SchedTask`):

Task	Rate (points / turn)	Key code element
FELL a tree	$0.8 - (d + 3 \cdot \text{manh}(\text{tree}, \text{enemy_shack})) \cdot 0.005$ — a <i>denial metric</i> , not a yield metric (see §3.1)	<code>fell_base</code> , <code>SchedTask::Fell</code>
BANK carried load	<code>carried pts / (walk home + 1)</code> — offered only when full , or with 1000× urgency when the clock barely covers the trip (endgame banking)	<code>SchedTask::Bank</code> , <code>turns_rem ≤ eta+2</code>
HARVEST ripe fruit	$\min(\text{fruits}, \text{harvest}, \text{free}) / (\text{travel} + 1 + \frac{1}{2} \cdot \text{return})$	<code>SchedTask::Harvest</code>
PRINT a crop	<code>6 / travel</code> to plant; <code>3</code> to PICK a seed at the shack (\$4)	<code>SchedTask::Print(cell, species)</code> , <code>PickSeed</code>
ORCHARD plum	<code>10 / travel</code> — water-adjacent base cell, max 2 (plum funds every troll's speed cost)	<code>SchedTask::Orchard</code>
MINE iron	<code>3 / travel</code> , only while iron blocks the next chopper	<code>SchedTask::Mine</code> , <code>need_iron</code>

3.1 Denial camping — the discovery that beats the boss

The fell rate deliberately **ignores wood yield** and prices a tree by *closeness to the enemy shack* (the classic metric `d + 3·manh(tree, oppShack)` from the earlier heuristic bot, re-expressed as a rate). Because that rate (≈ 0.8) is *below* a full load's bank rate but *above* everything else, a chopper that fills up far from home keeps felling at zero extraction — **permanently starving the opponent's tree supply** — and only banks when passing near home or at the endgame. Wood is worth 4 pts, but a tree denied to the boss costs it up to 16 pts + future fruit. Measured: this "camper" beats the boss-script model at **87.5%** where the yield-driven "cycler" variant (`SB_FELL_FREE=1`) manages 67%. From **80 turns before the end** the gate flips and fells require free carry — denial value decays to zero at the buzzer (`need_free = turns_rem ≤ 80`).

3.2 Training plan

Greedy, inherited from the proven heuristic ancestor: build up to **two fast cheap choppers** (2,2,0,2) as soon as affordable (speed wins the race to contested trees; carry 2 keeps them cheap — copying the boss's expensive carry-4 chopper measurably drains the economy), otherwise the cheapest affordable harvester of (2,2,2,0) / (1,2,2,0) / (1,1,1,0), capped at 4 trolls. Constants `MB_CHOPPER`, `MB_HARVESTERS` in `main.rs`.

4. The economy: wood printing and water crops

Fruit is seed capital; wood is income. Surplus fruit is planted near the base, grown fast beside water, and felled young — a self-sustaining "wood printer" worth ~8 pts per cycle. Decoded from the replays of the bots that were beating us (aRi: 67 wood = 268 of its 275 points; logiqub: 76 wood = 304 of 308).

- **Species follows the spot** (from logiqub's crop system): on a water-adjacent cell plant **APPLE** (grows a size per 2 ticks, and its high health makes it unprofitable for the enemy to steal) or **PLUM**; on dry cells plant **BANANA** (fastest dry grower, and worthless for training — pure surplus). Plum/apple are training currency, so they are only spent above a **reserve of 8** banked units (`SB_CROP_RES`).
- **PICK-based feeding:** harvesters PICK a seed back out of the bank at the shack when a plantable base spot exists — the printer never starves waiting for a lucky carry.
- The plum **orchard** (max 2, water-adjacent) keeps the troll-training pipeline funded; orchard duty outranks printing for a plum-carrying troll.

5. Correctness armour (each one converted real losses)

Mechanism	Failure it kills	Where
Endgame banking: any carrier whose remaining clock barely covers the walk home + DROP abandons its task and banks	carried goods scoring 0 at turn 300 (4 pts per stranded wood)	Bank urgency term
Wedge fix: full troll standing on the (unwalkable) shack cell targets the nearest walkable drop cell	starter freezing for 300 turns after mining turn-1 iron beside the shack	drop-cell fallback in Bank/legacy <code>decide()</code>
Plant-spot gate: PICK is only offered when a plantable base cell exists	the pick/drop livelock family: a carry-1 starter picking a banana, failing to plant, banking it back — forever (a real game lost 21-148 with 130 wasted turns; the user-reported replay showed 119 PICKs + 129 DROPs)	spot computed for both arms of the printer
Single-ferry gate: no PICK while another troll already carries a seed		PickSeed condition
PICK cooldown (12 turns): a recent PICK no longer carried means the plant failed — cool down instead of looping		SB_PICKED thread-local
Dry-cell parking: idle trolls park on non-water-adjacent cells so they never squat the plantable spot		idle fallback in the emit phase
Contention guard: idle pre-positioning avoids cells already assigned or occupied this turn	two trolls MOVE-blocking each other on one target forever (3-troll jam, real loss 61-180)	anticipate filters on <code>taken</code>

The livelock fixes alone moved the simulator win rates from 83.9%/82.7% to **87.5%/87.5%** on the two boss models, and the arena rank from ~40 to **1**.

6. Methodology: how changes are validated

- Fast local benchmark** — `bench A B N` plays `N` seeds × both seats in parallel (`rust/src/bin/bench.rs`) on a referee-faithful simulator (`rust/src/game/engine.rs` , validated against real replays).
- Two-model rule:** a change is accepted only if it helps (or holds) against *both* `silverboss` and `scriptboss`. Anchor-calibrated: the models correctly order historical bots by their real win rates where a single model inverted them.

3. **Real batches before defaults for risky classes.** Three *transfer-failure modes* were measured, where the sim said "better" and the arena said "worse":

1. Overfitting one boss model (v1.0.4: 90.5% sim, 33% real). **2.** Troll-spec/economy changes interacting with map-economy calibration (v1.0.9 "mower": best-ever on both models, 25% real). **3.** Opponent-conditional triggers under-represented by both models (v1.1.3 raid gate: +1.7pp on models, −5 rating points in the arena).

4. **Arena replay decoding** (pure HTTP, no browser): `findLastBattlesByAgentId` → `gameResult/findByGameId` exposes both players' raw command streams, per-turn inventories and summaries. This pipeline found every major leak and decoded the elite engines. Submissions are verified by a **new agentId** on the leaderboard, never by the submitter's exit code.

7. Result trajectory (one day, 2026-07-02)

Version	Change	Arena outcome
v1.0.1 (previous day)	heuristic wood-race + denial	rank 42 → slid to 134 as the field improved
v1.0.6-tempo	endgame banking, cheap hp0 chopper, ripeness anticipation, water orchard	rank ~55
v1.0.8-woodprinter	woodfarm + banana printer (decoded from aRi)	rank ~48-70, real avg score 152→164
v1.1.0/v1.1.1-sched	scheduler architecture , camper discovery, first livelock fixes	rank 18 → 10
v1.1.2-sched	anticipate-contention fix, single-ferry gate	rank 2 (score 22.64)
v1.1.4-crops	species-aware water crops (decoded from logiqub)	mid-placement when superseded
v1.1.5-nolock	PICK cooldown + dry parking (user-reported replay)	rank 1 / 681, score 23.95 ; promotion watch vs Boss 4

8. File map

Element	Purpose
<code>rust/src/main.rs</code> — <code>decide_sched()</code> , <code>SchedTask</code> , <code>MB_*</code> consts	the live single-file CodinGame bot (compiles with <code>rustc --edition 2021 -O</code>)
<code>rust/src/strategies/sched_bot.rs</code>	simulator mirror of the live bot; env knobs <code>SB_*</code> for sweeps
<code>rust/src/strategies/script_boss.rs</code> / <code>silver_boss.rs</code>	the two Boss-4 sparring models (§2, §6)
<code>rust/src/game/engine.rs</code> , <code>mapgen.rs</code>	referee-faithful rules engine + Silver-calibrated map generator
<code>rust/src/bin/bench.rs</code> , <code>diag.rs</code> , <code>trace.rs</code> , <code>mapstat.rs</code>	parallel benchmark, score composition, per-turn traces
<code>cgauto/</code> (<code>run_games.py</code> , <code>submit.py</code> , <code>submissions/</code>)	real-CG automation: validation batches, arena submission, frozen artifacts
<code>cgauto/HANDOFF.md</code> , <code>docs/silver-experiment-log.md</code>	the full experiment record every number in this document comes from

Generated 2026-07-02 late evening. Numbers: simulator win rates are 1000-seed × 2-seat benchmarks; arena figures are CodinGame leaderboard reads (public API). tass = the arena account name.